

Lecture 2

Introduction



About Me

- CSE Teaching Faculty
- Academic interests:
 - CS Education
 - Interactive Program Visualization
 - Principles of Software Design
- Other interests:
 - Spicy foods
 - Gardening
 - Cats
 - Escape games



Syllabus and Course Schedule

- The syllabus and course schedule are available via:

eecs498-software-design.org

Project Structure

5 projects. Incremental development of a single, large-scale codebase.

All project due dates are on **Fridays**.

Each project involves several sequential components:

- **Phase 1: Initial Writing** (two weeks)
Deliverables: Feature Demo in Lab, Project Deliverables via GitHub 11:59pm
- [staff] Initial Review (one week)
- **Phase 2: Revision** (two weeks)
Deliverables: Feature Demo in Lab, Project Deliverables via GitHub 11:59pm
- [staff] Final Grading (one week)



Project Timelines

	Fri 1/23				Fri 2/6		Fri 2/13	Fri 2/20	Midterm	Break	Fri 3/13	Fri 3/20	Fri 3/27	Fri 4/3	Fri 4/10	Fri 4/17		Final
Student	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
Project 1		Write	Write		Revise	Revise												
Project 2				Write	Write		Revise			Revise								
Project 3						Write	Write				Revise	Revise						
Project 4										Write	Write		Revise	Revise				
Project 5												Write	Write		Revise			
Staff																		
Initial Review				P1		P2				P3		P4		P5				
Final Grading							P1				P2		P3		P4	P5		

- Project timelines overlap with each other.
- Initial writing for the next project begins immediately after the last.
- Consider when to interleave revisions.

Projects

- **Overall Theme**

Collaborative Puzzle Solving System

- **Project 1 (Individual) – Released 2nd week, Due Fri Jan 23**

Implement a terminal based puzzle game. Choose one:

- Sudoku
- Minesweeper
- Spelling Bee

- **Remaining Projects (Group of 3)**

Combine codebases from P1 to for a puzzle game collection.

Incrementally add features and revise design.



Project Groups

- Work in groups of 3 for most projects.
- Exception: Initial writing phase of P1 is individual.
- The three options for P1 implementation combine to form the initial codebase for P2.
 - Aim to work with others who are completing different options.
- If you are not able to find a group, please reach out to us.

Project Repositories and Submission

- We'll provision GitHub repositories for you.
- Submissions for initial deliverables and revisions are made via a pull request and associated commit.
- More information in project specifications.

Project Grading and Feedback

- **Project Feature Demos (3% overall grade)**

Project specification will include details of what to demo.

- **Initial Review + Final Grading (3% + 3% overall grade)**

Grade according to rubric:

0 - Missing

1 - Insufficient

2 - Growth Needed

3 - Exemplary

Initial review more lenient, final grading more strict.



Typescript Introduction

Type Systems

- A **type** defines a set of possible values.
 - e.g. **string**: "", "cat", "hello world!"
 - e.g. **boolean**: true, false
 - e.g. **Person**: {"james", 2, 110, 0xE5A4}
 - **Static Typing**:
Variables have a fixed type.
 - **Dynamic Typing**:
A variable may refer to different types.
The type is on the value only.
-
- The job of a type system:
 - What **operations** are available?
 - Which **conversions*** are allowed?
 - **Strong Typing**
Few, if any, implicit conversions.
 - **Weak Typing**
Eh, just do whatever.

*We usually think of this first in terms of **implicit conversions**, sometimes called coercions. Explicit casts may be used to circumvent the usual behavior of the type system (and sometimes its safety guarantees).



Type Systems

Where is C++?

- A **type** defines a set of possible values.
 - e.g. **string**: "", "cat", "hello world!"
 - e.g. **boolean**: true, false
 - e.g. **Person**: {"james", 2, 110, 0xE5A4}
- **Static Typing**:
Variables have a fixed type.
- **Dynamic Typing**:
A variable may refer to different types.
The type is on the value only.
- The job of a type system:
 - What **operations** are available?
 - Which **conversions*** are allowed?
- **Strong Typing**
Few, if any, implicit conversions.
- **Weak Typing**
Eh, just do whatever.

*We usually think of this first in terms of **implicit conversions**, sometimes called coercions. Explicit casts may be used to circumvent the usual behavior of the type system (and sometimes its safety guarantees).



Type Systems

Where is Javascript?

Where is Typescript?

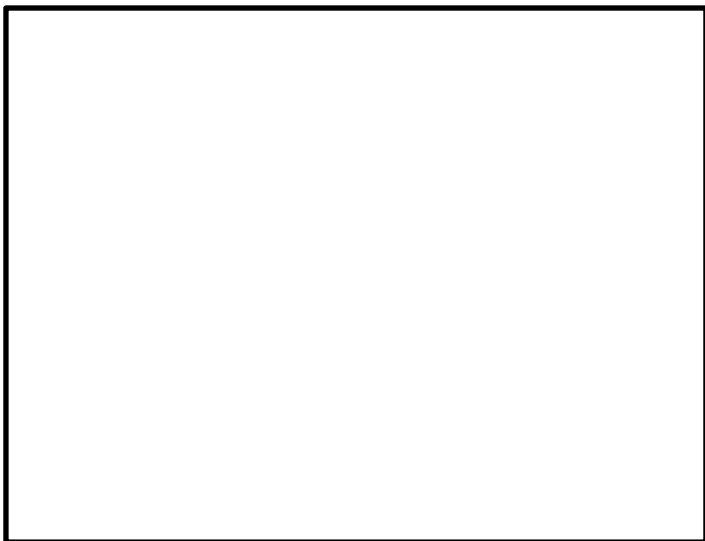
- A **type** defines a set of possible values.
 - e.g. **string**: "", "cat", "hello world!"
 - e.g. **boolean**: true, false
 - e.g. **Person**: {"james", 2, 110, 0xE5A4}
- **Static Typing**:
Variables have a fixed type.
- **Dynamic Typing**:
A variable may refer to different types.
The type is on the value only.
- The job of a type system:
 - What **operations** are available?
 - Which **conversions*** are allowed?
- **Strong Typing**
Few, if any, implicit conversions.
- **Weak Typing**
Eh, just do whatever.

*We usually think of this first in terms of **implicit conversions**, sometimes called coercions. Explicit casts may be used to circumvent the usual behavior of the type system (and sometimes its safety guarantees).

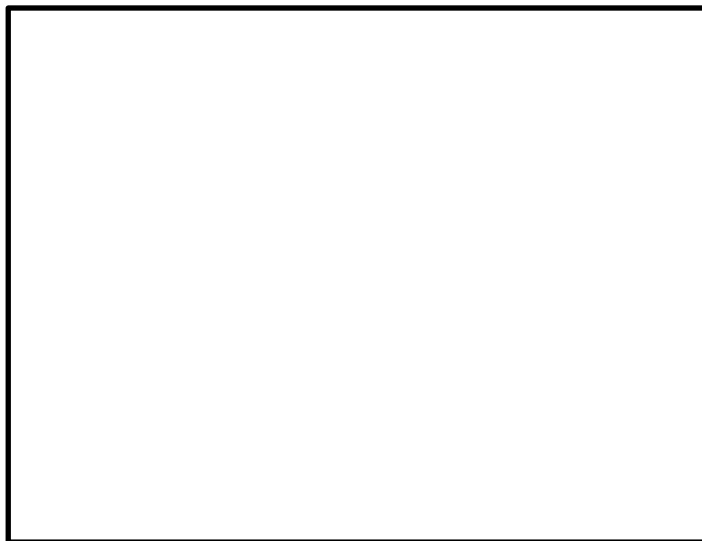


Value Semantics vs. Reference Semantics

C++



Typescript



Typescript Fundamentals

DEMO



Grading Policies

- **Lecture**

You may miss up to 4 lectures and still earn full credit.

- **Lab**

You may miss up to 2 labs and still earn full credit.

- **Project Checkpoints**

Ensure at least one member of your group is present at lab for checkpoints. Of course, the ideal is that everyone should attend.



Academic Integrity

- You're encouraged to collaborate.
- Don't submit others work as your own.
- Don't do others work for them.
- Don't share copies of code or grant access to your repo.
- Earnestly engage with your project team.



AI Coding Tools

- AI usage is allowed, but you must document how and where it is used. This will help us give relevant feedback.
- We recommend against using agentic coding tools.
- Tools like Copilot that work more as an “autocomplete” are probably maybe OK, but keep an eye on your own fluency.
- When using AI, use it to build your own knowledge and capabilities, not to replace yourself.



Break Time